

UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
CURSO DE CIÊNCIA DA COMPUTAÇÃO

MATHEUS THUROW
RODRIGO NICOLA BERARDI JUNIOR
EDUARDO

PROCESSAMENTO CONCORRENTE DE REQUISIÇÕES A UM BANCO DE
DADOS SIMULADO NO WINDOWS

Itajaí
2026

PROCESSAMENTO CONCORRENTE DE REQUISIÇÕES A UM BANCO DE DADOS SIMULADO NO WINDOWS

Relatório apresentado à disciplina de Sistemas Operacionais do curso de ciência da computação, da Universidade do Vale do Itajaí, como parte da Avaliação M1 de 2026/2.

Professor: Michel Douglas Cabral Alves.

RESUMO

Este trabalho apresenta um sistema em C++ que simula o processamento de requisições a um banco de dados no Windows. A solução utiliza processos cliente e servidor separados, comunicação por named pipe, um pool de threads e mutexes para proteger as estruturas compartilhadas. Foram implementadas as operações INSERT, SELECT, UPDATE e DELETE sobre um vetor de registros. A validação funcional verificou essas operações, o tratamento de entradas inválidas e a disputa por um mesmo identificador. A avaliação de desempenho comparou pools de uma, duas, quatro e oito threads, com quatro clientes e lotes de 100, 1.000 e 5.000 inserções, em cinco repetições por cenário. As 60 medições finais totalizaram 122.000 inserções verificadas sem perdas, duplicações ou respostas inesperadas. Para 5.000 inserções, as medianas foram 0,2620 s com uma thread e 0,3037 s com quatro. Os resultados demonstram funcionamento correto nos cenários testados, mas não evidenciam ganho consistente com o aumento do pool. A serialização do banco, a comunicação e a escrita dos logs são fatores compatíveis com esse comportamento.

Palavras-chave: comunicação entre processos; threads; exclusão mútua; banco de dados simulado; Windows.

ABSTRACT

This report presents a C++ system that simulates database request processing on Windows. The solution uses separate client and server processes, named-pipe communication, a thread pool, and mutexes to protect shared data structures. INSERT, SELECT, UPDATE, and DELETE operations were implemented over a vector of records. Functional tests checked these operations, invalid inputs, and concurrent attempts to insert the same identifier. Performance experiments compared pools of one, two, four, and eight threads, with four clients and batches of 100, 1,000, and 5,000 insertions, using five repetitions per scenario. The 60 final measurements covered 122,000 insertions with no missing records, duplicates, or unexpected responses in the verified logs. For 5,000 insertions, median batch times were 0.2620 seconds with one thread and 0.3037 seconds with four. The tested scenarios behaved correctly, but increasing the pool did not yield a consistent performance improvement. Serialized database access, communication, and logging are factors consistent with this result. The measurements describe the complete batch, including synchronization and shutdown, rather than the latency of an individual request.

Keywords: interprocess communication; threads; mutual exclusion; simulated database; Windows.

SUMÁRIO

1 INTRODUÇÃO	5
1.1 Enunciado e requisitos	5
1.2 Processos threads e sincronização	6
2 ARQUITETURA E IMPLEMENTAÇÃO	7
2.1 Trechos principais da implementação.....	8
3 METODOLOGIA EXPERIMENTAL	9
4 RESULTADOS E DISCUSSÃO.....	10
4.1 Distribuição dos tempos	11
4.2 Validação funcional e utilização do pool.....	12
4.3 Discussão e limitações	12
5 CONCLUSÃO	13
REFERÊNCIAS	14
APÊNDICE A – INSTRUÇÕES DE EXECUÇÃO	15

1 INTRODUÇÃO

O atendimento de requisições por múltiplos clientes exige coordenação entre unidades de execução. Quando diferentes threads acessam uma mesma tabela, operações como verificar a existência de um identificador e inserir um registro precisam ocorrer de forma protegida para evitar condições de corrida.

O objetivo deste projeto é aplicar os conceitos de IPC, threads e sincronização em uma implementação simples e executável, conforme a proposta da Avaliação M1 (1). Além de verificar o comportamento funcional, o trabalho compara o tempo de atendimento de lotes com diferentes tamanhos de pool. O estudo distingue o uso correto da concorrência de um eventual ganho de desempenho.

1.1 Enunciado e requisitos

A proposta consiste em simular um gerenciador de requisições a um banco de dados. Um processo cliente envia comandos por IPC real a um processo servidor, que mantém um pool de threads para atendê-los. As threads compartilham uma estrutura que representa o banco e devem utilizar mutex ou semáforo para preservar sua integridade. Devem ser suportadas inserção, consulta, atualização e exclusão de registros, com respostas por outro canal IPC ou por arquivo de log (1).

A implementação utiliza dois executáveis, cliente.exe e servidor.exe. O banco é um vetor em memória, contendo ID e nome. As respostas das operações são gravadas em servidor.log e exibidas no terminal do servidor. O projeto não implementa um interpretador SQL completo; os comandos seguem uma sintaxe textual reduzida.

Quadro 1 – Correspondência entre requisitos e implementação

Requisito	Implementação
Processos separados	cliente.exe e servidor.exe, cada um com sua função main.
IPC real	Named pipe do Windows, criado com CreateNamedPipeA.
Pool de threads	Threads criadas com CreateThread dentro de servidor.cpp.
Proteção dos dados	Mutex do banco criado com CreateMutexA; fila e log têm mutexes próprios.
Sinalização de trabalho	Semáforo criado com CreateSemaphoreA.
Operações e respostas	INSERT, SELECT, UPDATE e DELETE; resultado em log.

Fonte: elaboração dos autores com base na implementação e nos dados experimentais (2026).

1.2 Processos threads e sincronização

Um processo reúne os recursos necessários à execução de um programa. As threads de um mesmo processo compartilham seu espaço de endereçamento e seus recursos (2). Neste projeto, a memória do banco pertence somente ao servidor; o cliente não a acessa diretamente e envia suas solicitações pelo named pipe. Esse mecanismo permite comunicação entre processos independentes (3).

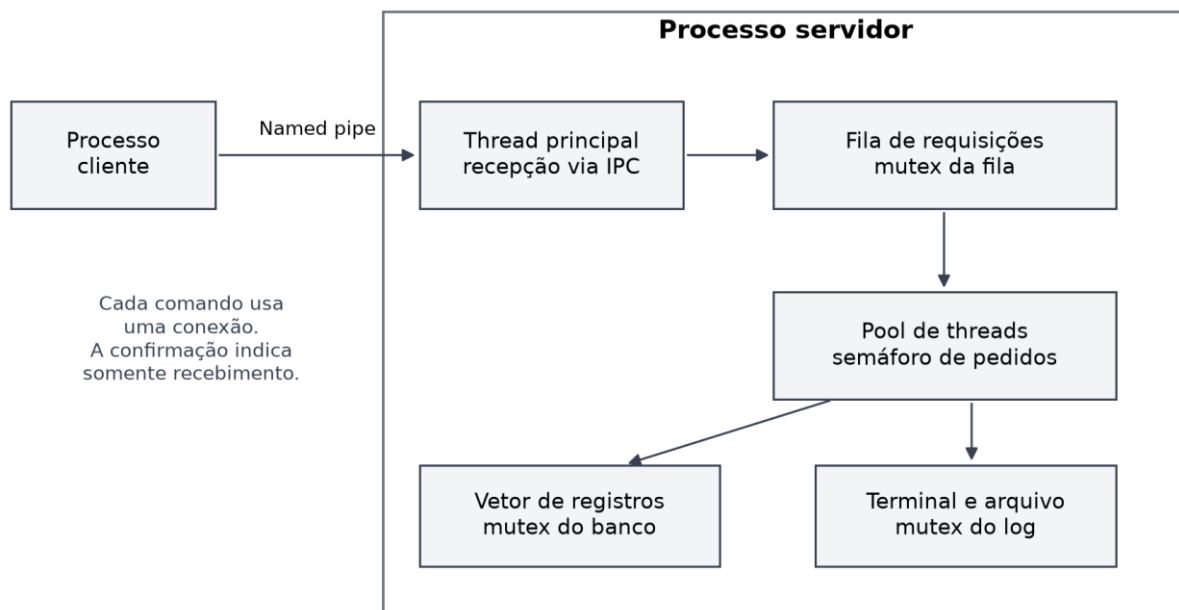
O mutex concede propriedade exclusiva a uma thread por vez (4). Já o semáforo mantém uma contagem que é reduzida quando uma espera é satisfeita e aumentada quando há liberação (5). Na solução, o mutex protege a tabela, enquanto o semáforo sinaliza a existência de pedidos na fila. Essa separação evita que as threads consultem continuamente uma fila vazia.

Foram escolhidas APIs nativas do Windows equivalentes em finalidade às primitivas citadas na proposta: `CreateThread` para criação, `CreateMutexA` para exclusão mútua e `CreateSemaphoreA` para sinalização. A implementação preserva os conceitos de threads e sincronização, mas não utiliza a interface POSIX. O pool é construído pelo programa, sem utilizar o serviço de thread pool pronto do Windows.

2 ARQUITETURA E IMPLEMENTAÇÃO

A thread principal recebe cada comando pelo named pipe e o coloca na fila protegida. O semáforo sinaliza trabalho disponível. Uma thread do pool retira o pedido, libera a fila e executa a operação no banco. Outro mutex protege a gravação da resposta. A Figura 1 apresenta esse fluxo.

Figura 1 – Organização dos processos e das estruturas compartilhadas



Fonte: elaboração dos autores com base na implementação e nos dados experimentais (2026).

A confirmação indica somente recebimento; o resultado fica no log. Como as threads podem concluir pedidos em outra ordem, uma operação dependente deve ser enviada após o resultado da anterior. O mutex não garante ordem de conclusão.

Os IDs são inteiros não negativos. Os nomes podem conter espaços e têm limite de 49 bytes. O comando PARAR interrompe a recepção e permite concluir os pedidos já recebidos antes de fechar o servidor. O banco é volátil: seus registros não persistem após o encerramento.

2.1 Trechos principais da implementação

O pool é criado uma vez em servidor.cpp e reutilizado. O servidor aceita de uma a 64 threads, com quatro como padrão. Os números passados como argumentos permanecem válidos até o término do pool.

Código 1 – Criação do pool em servidor.cpp

```
for (int i = 0; i < quantidade_threads; ++i) {
    numeros[i] = i + 1;
    threads[i] = CreateThread(nullptr, 0, atender,
                              &numeros[i], 0, nullptr);
    if (!threads[i]) break;
    ++criadas;
}
```

CreateThread recebe a função que a thread executará e seu argumento (6). Na rotina atender, o semáforo controla a espera por trabalho. O bloco delimitado abaixo protege somente a retirada da fila; a execução da operação ocorre depois de sua liberação.

Código 2 – Espera e retirada de um pedido na rotina atender

```
WaitForSingleObject(tem_requisicao, INFINITE);
std::string pedido;
{
    GuardaMutex guarda(mutex_fila);
    if (fila.empty() && encerrando) break;
    if (fila.empty()) continue;
    pedido = fila.front();
    fila.pop();
}
std::string resposta = executar_requisicao(pedido);
```

GuardaMutex adquire o mutex no construtor e o libera no destrutor. Assim, a saída do bloco libera a proteção. Em banco.cpp, o mesmo recurso cobre toda a operação, incluindo a busca do ID. SELECT também adquire o mutex, pois uma escrita simultânea poderia modificar o vetor durante a leitura.

Código 3 – Proteção das operações em banco.cpp

```
std::string executar_requisicao(const std::string& texto) {
    GuardaMutex guarda(mutex_banco);
    return executar(texto);
}
```

Com PARAR, o servidor sinaliza o encerramento e acorda as threads. O pool conclui a fila; WaitForMultipleObjects aguarda todas as threads antes do fechamento dos handles.

3 METODOLOGIA EXPERIMENTAL

Quadro 2 – Ambiente das simulações de 19 de setembro de 2026

Item	Configuração
Sistema	Windows 11, versão 10.0.26200
Processador	Intel Core i5-12450HX; 8 núcleos e 12 processadores lógicos
Memória	15,73 GiB utilizáveis pelo sistema operacional
Compilação	MinGW GCC 6.3.0; C++11; -static; sem opção explícita -O
Automação	Python 3.12.14; script testes/simular.py
Cenários	1, 2, 4 e 8 threads; 100, 1.000 e 5.000 inserções
Clientes e repetições	4 clientes; 5 repetições por combinação; 60 medições

Fonte: configuração registrada em hardware.json e dados.json da rodada final (2026).

Cada execução iniciou um servidor com banco vazio. Os IDs foram distintos, de zero a $N-1$, distribuídos entre quatro clientes. O tamanho do lote corresponde ao total enviado pelos quatro processos. A carga de desempenho contém somente INSERT; as demais operações foram verificadas separadamente no teste funcional. Não foram adicionados atrasos artificiais para favorecer uma configuração.

Antes das medições, foi realizado um aquecimento de 1.000 inserções para cada tamanho de pool. A ordem dos 60 ensaios foi embaralhada com semente 20260919. Nenhuma repetição da rodada final foi excluída. Uma rodada preliminar coincidiu com a instalação da biblioteca de gráficos e foi preservada apenas como registro, sem participar das estatísticas.

O tempo foi medido por `time.perf_counter()`, desde o despacho dos envios até a saída do servidor após PARAR. A medida inclui IPC, operações, sincronização, logs, coordenação Python e encerramento. As chamadas de criação dos processos ocorreram antes do cronômetro, embora parte da inicialização dos clientes possa ter ocorrido depois. A validação do log foi realizada fora do intervalo medido.

O log permaneceu ativo, com flush por resposta. A saída do servidor foi redirecionada a um arquivo e a saída normal dos clientes foi descartada. Portanto, o tempo apresentado é do lote completo nessas condições; não é latência individual nem tempo exclusivo do banco. As estatísticas usadas são mediana, média, desvio padrão amostral e faixa observada. A vazão é N dividido pela mediana, e a aceleração é a razão entre a mediana com uma thread e a mediana do cenário.

4 RESULTADOS E DISCUSSÃO

As 60 medições finais totalizaram 122.000 inserções. Todas apresentaram a quantidade esperada de respostas, IDs completos e distintos, correspondência entre ID e nome e códigos de saída iguais a zero. Os quatro aquecimentos adicionais, com 4.000 inserções, não estão incluídos na Tabela 1.

Tabela 1 – Tempos e vazão por cenário com cinco repetições

Lote	Threads	Mediana (s)	Média ± DP (s)	Mín. a máx. (s)	Vazão (req/s)	Acel. (vezes)
100	1	0,0204	0,0223 ± 0,0085	0,0149 a 0,0365	4.912	1,00
100	2	0,0207	0,0310 ± 0,0173	0,0146 a 0,0500	4.834	0,98
100	4	0,0221	0,0321 ± 0,0178	0,0141 a 0,0528	4.526	0,92
100	8	0,0207	0,0306 ± 0,0152	0,0184 a 0,0487	4.820	0,98
1.000	1	0,0575	0,0671 ± 0,0228	0,0498 a 0,1046	17.380	1,00
1.000	2	0,0541	0,0583 ± 0,0124	0,0499 a 0,0801	18.493	1,06
1.000	4	0,0720	0,0772 ± 0,0253	0,0534 a 0,1059	13.882	0,80
1.000	8	0,0671	0,0735 ± 0,0196	0,0543 a 0,1064	14.901	0,86
5.000	1	0,2620	0,2643 ± 0,0172	0,2471 a 0,2904	19.082	1,00
5.000	2	0,2928	0,2842 ± 0,0181	0,2594 a 0,3037	17.075	0,89
5.000	4	0,3037	0,2918 ± 0,0229	0,2659 a 0,3111	16.466	0,86
5.000	8	0,2968	0,2945 ± 0,0258	0,2607 a 0,3323	16.844	0,88

Fonte: dados.json e resumo.json da rodada simulacao_20260919_213111 (2026).

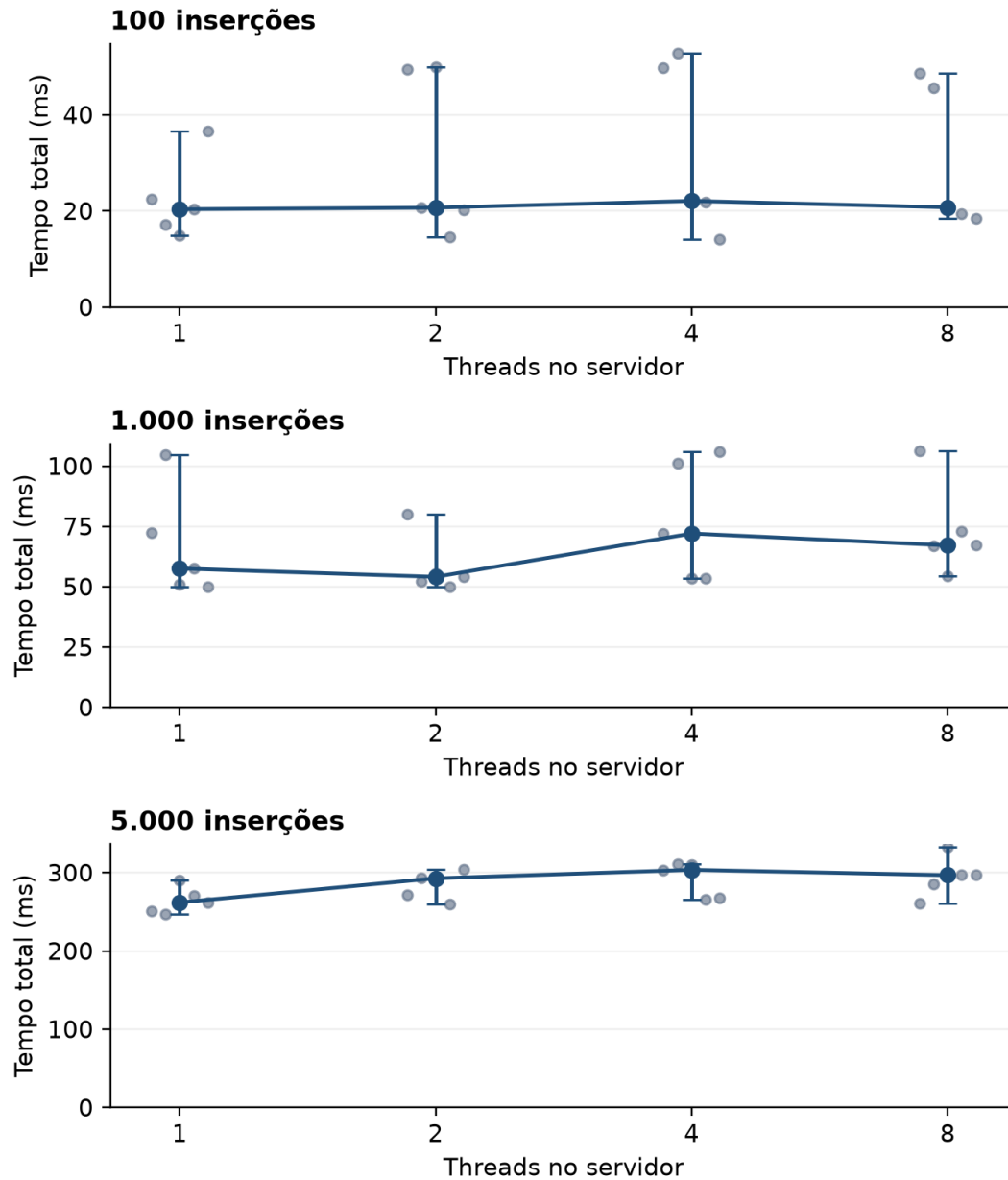
DP representa o desvio padrão amostral. Aceleração abaixo de 1 indica que a mediana foi maior que a obtida com uma thread para o mesmo lote. A faixa mínima e máxima descreve a dispersão observada e não corresponde a um intervalo de confiança.

No lote de 5.000 inserções, uma thread apresentou mediana de 0,2620 s e vazão de aproximadamente 19.082 requisições/s. Com quatro threads, a mediana foi de 0,3037 s, ou 15,9% maior, com vazão de cerca de 16.466 requisições/s. Essa comparação descreve a amostra medida, sem estabelecer uma vantagem universal de uma configuração. Esse comportamento

pode estar relacionado à sincronização necessária para o acesso à estrutura compartilhada, uma vez que o uso de mutex faz com que as operações realizadas na região crítica sejam executadas de forma sincronizada, limitando parte do ganho potencial obtido com o aumento do número de threads.

4.1 Distribuição dos tempos

Figura 2 – Tempos dos lotes conforme o número de threads



Fonte: elaboração dos autores a partir das 60 medições finais (2026).

A Figura 2 apresenta as cinco execuções de cada cenário. A linha representa a mediana e as barras indicam mínimo e máximo; não são intervalos de confiança. As escalas verticais são independentes. O tempo inclui IPC, logs e encerramento.

Observa-se que o aumento do número de threads não resultou necessariamente em redução do tempo de execução. Principalmente nos testes com maior número de inserções, os tempos permaneceram próximos ou apresentaram pequenas variações. Esse comportamento está relacionado ao acesso compartilhado ao banco de dados, que é protegido por mutex. Dessa forma, as operações realizadas na região crítica são sincronizadas, limitando o ganho de desempenho com o aumento do número de threads, mas garantindo a integridade dos dados e evitando condições de corrida.

4.2 Validação funcional e utilização do pool

O teste funcional executou INSERT, SELECT, UPDATE e DELETE, além de verificar ID duplicado, registro inexistente, ID negativo, ID com letras, nome ausente e argumentos extras. Na disputa de 12 clientes pelo mesmo ID, uma inserção foi aceita e 11 foram rejeitadas. Também foram verificadas 20 inserções seguidas de PARAR e a falha de conexão após o encerramento do servidor.

Nos cinco lotes de 5.000 inserções com quatro threads, foram registradas 6.244 operações na thread 1, 6.255 na thread 2, 6.247 na thread 3 e 6.254 na thread 4, totalizando 25.000. As contagens confirmam a participação de todas as threads nessa configuração. Elas não demonstram execução simultânea dentro do banco, pois sua seção crítica é exclusiva.

4.3 Discussão e limitações

Para 1.000 inserções, duas threads tiveram mediana ligeiramente menor, mas as faixas observadas se sobrepõem. As diferenças não são consistentes nos demais volumes.

O aumento do pool não produziu redução consistente do tempo. O acesso ao vetor ocorre sob um único mutex, incluindo a busca sequencial do identificador. Dessa forma, as operações no banco são serializadas. A recepção atende uma conexão de cada vez, e o log possui sincronização e flush por resposta. Acrescentar threads mantém essas etapas e adiciona coordenação. Esses fatores são compatíveis com o resultado, mas o experimento não isolou a contribuição de cada um.

As medições têm limites: cinco repetições por cenário, lotes curtos, uma única máquina, banco inicialmente vazio, carga apenas de inserção e compilação sem otimização explícita. O computador não foi isolado de toda atividade de fundo. Não foram coletados uso de CPU, consumo de memória, percentis de latência ou intervalos de confiança. Os resultados não devem ser generalizados para bancos persistentes ou cargas intensivas de cálculo.

A solução também possui limites funcionais deliberados: o banco não é persistente, a fila não tem tamanho máximo e os clientes usam um protocolo simples. O programa não oferece transações compostas nem garante a ordem de conclusão de pedidos dependentes. Essas restrições mantêm o projeto didático e ajudam a delimitar os resultados apresentados.

5 CONCLUSÃO

O sistema implementou comunicação real entre processos, um pool de threads no servidor e proteção das estruturas compartilhadas por mecanismos do Windows. Os testes funcionais e as simulações verificaram as operações nos cenários avaliados, sem perdas ou duplicações nas 122.000 inserções medidas.

A avaliação mostrou que concorrência correta não implica aceleração. Para a carga utilizada, ampliar o pool não trouxe ganho consistente. Como continuidade, podem ser avaliados lotes maiores, mais repetições e cargas de consulta e atualização, com preparação dos dados e controle das dependências. A redução das seções críticas e a mudança da estratégia de log exigiriam novas medições antes de qualquer conclusão sobre desempenho.

REFERÊNCIAS

- 1 UNIVERSIDADE DO VALE DO ITAJAÍ. **Avaliação M1: IPC, Threads e Paralelismo**. [S. l.]: Univali, 2026. Material da disciplina de Sistemas Operacionais, 2026/2. Arquivo Trabalho M1_2026_2_propost.pdf.
- 2 MICROSOFT. **About Processes and Threads**. Microsoft Learn, 14 jul. 2025. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads>. Acesso em: 19 set. 2026.
- 3 MICROSOFT. **Named Pipes**. Microsoft Learn, 7 jan. 2021. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes>. Acesso em: 19 set. 2026.
- 4 MICROSOFT. **Mutex Objects**. Microsoft Learn, 7 jan. 2021. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/sync/mutex-objects>. Acesso em: 19 set. 2026.
- 5 MICROSOFT. **Semaphore Objects**. Microsoft Learn, 7 jan. 2021. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/sync/semaphore-objects>. Acesso em: 19 set. 2026.
- 6 MICROSOFT. **Creating Threads**. Microsoft Learn, 17 jul. 2026. Disponível em: <https://learn.microsoft.com/en-us/windows/win32/procthread/creating-threads>. Acesso em: 19 set. 2026.

APÊNDICE A – INSTRUÇÕES DE EXECUÇÃO

O projeto contém fontes, dois executáveis Windows, script de compilação, testes, dados brutos e logs. Para compilar, é necessário g++ no PATH. O arquivo compilar.bat gera cliente.exe e servidor.exe. Em dois terminais na pasta do projeto, execute os comandos a seguir.

```
.\compilar.bat
.\servidor.exe 4      # primeiro terminal
.\cliente.exe         # segundo terminal
python testes/testar.py
python testes/simular.py
```

Os dois últimos comandos devem ser executados depois de encerrar qualquer servidor manual. Cada script inicia o próprio servidor. As medições analisadas estão em resultados/simulacao_20260919_213111. O arquivo dados.json preserva os tempos sem arredondamento, a ordem dos ensaios e hashes SHA-256 dos fontes e executáveis; resumo.json reúne as estatísticas.

Quadro 3 – Comandos aceitos pelo sistema

Comando	Exemplo	Comportamento
INSERT	INSERT 1 Ana Silva	Insere um registro; rejeita ID existente.
SELECT	SELECT 1	Consulta o registro pelo ID.
UPDATE	UPDATE 1 Maria Silva	Altera o nome de um registro existente.
DELETE	DELETE 1	Remove o registro pelo ID.
SAIR / PARAR	SAIR ou PARAR	Fecha o cliente ou encerra o servidor, respectivamente.

Fonte: elaboração dos autores com base na implementação e nos dados experimentais (2026).

Repositório público do projeto: [\[Link do Repositório\]](#).